

Autonomously evolving classifier TEDAClass



Dmitry Kangin^{a,*}, Plamen Angelov^a, José Antonio Iglesias^b

^a Data Science Group, Computing & Communications, Lancaster University, UK

^b Carlos III University of Madrid, Spain

ARTICLE INFO

Article history:

Received 28 November 2015

Revised 16 March 2016

Accepted 12 May 2016

Available online 24 May 2016

Keywords:

Classifiers

Evolving systems

TEDA

Fuzzy systems

ABSTRACT

In this paper we introduce a classifier named TEDAClass (Typicality and Eccentricity based Data Analytics Classifier) which is based on the recently proposed AnYa type fuzzy rule based system. Specifically, the rules of the proposed classifier are defined according to the recently proposed TEDA framework. This novel and efficient systematic methodology for data analysis is a promising addition to the traditional probability as well as to the fuzzy logic. It is centred at non-parametric density estimation derived from the data sample. In addition, the proposed framework is computationally cheap and provides fast and exact per-point processing of the data set/stream. The algorithm is demonstrated to be suitable for different classification tasks. Throughout the paper we give evidence of its applicability to a wide range of practical problems. Furthermore, the algorithm can be easily adapted to different classical data analytics problems, such as clustering, regression, prediction, and outlier detection. Finally, it is very important to remark that the proposed algorithm can work “from scratch” and evolve its structure during the learning process.

© 2016 Elsevier Inc. All rights reserved.

1. Introduction

Nowadays, the classical problems of machine learning are widely studied. However, many architecturally different algorithms still suffer from a number of limitations. If we take into account how they work, we can consider three different kinds of algorithms: offline, incremental and online learning. The offline algorithms are trained once, and then do not admit any modifications to the classifier parameters or structure. Hence, the performance of these algorithms is only restricted to the patterns observed during the training stage. Furthermore, the incremental algorithms admit modifications of (some of) the parameters, but they generally demand previous data samples (all or some part) to be stored in the memory. They do not allow modification of the classifier structure, and very often they are not sufficient for practical needs. Finally, the algorithms, which implement online learning, do not need to memorise all previous training samples. This kind of algorithms are usually computationally efficient, recursive procedure, and applicable for online real-time applications.

However, none of these three types of algorithms involve any kind of adaptation of the structure to take into account the newly emerging or dynamically evolving data patterns which were not present in the initial training data set/stream. For this reason, a recently emerged branch of machine learning aiming to address such problems: Evolving Intelligent Systems (EIS) [4]. The algorithm developed in this work is based on the paradigm of such systems that is a distinguishable feature of the proposed algorithm amongst the well-studied techniques. One of the most renowned machine learning challenges is the classification problem. Some of the versions of the problem statement are domain-specific, while others are defined

* Corresponding author. Tel.: +44 7437499029.

E-mail addresses: d.kangin@lancaster.ac.uk (D. Kangin), p.angelov@lancaster.ac.uk (P. Angelov), jiglesia@inf.uc3m.es (J.A. Iglesias).

generally, and then adapted for a particular problem. Hereafter we provide a classifier which does not have any user- or problem-specific parameter.

One of the most renowned machine learning challenges is the classification problem. Some of the versions of the problem statement are domain-specific, while others are defined generally, and then adapted for a particular problem. Hereafter we provide a classifier which does not have any user- or problem-specific parameter.

Define some object set Ω , and a finite space of object classes, referred as C . Then define a subset, $\Omega_L \subset \Omega$ called learning set, for each of the following functions defined as: $F_L: \Omega_L \rightarrow C$. The problem is to build a function $F: \Omega \rightarrow C$, approximating the function F_L on the set Ω_L . The assumption is that F will be a good mapping for further objects, Ω_V where the index V denotes validation. $\Omega_V \cap \Omega_L = \emptyset$, $\Omega_V, \Omega_L \subset \Omega$.

In this paper, we address the problem of a general purpose classifier, which needs to meet the following requirements:

- *Ability of incremental learning*: the classifier should be one-pass. Thus, each sample is proceeded once and there is no need to re-learn any previous samples once the new training data appear.
- *Online design*: the classifier should proceed the samples at the time of their arrival, the algorithm should not rely on memorised context of all data samples. There is no need to store all the given samples, but only some descriptor data of them, ideally with fixed size upper bound.
- *Evolving structure*: the classifier should be able to start learning “from scratch” and its parameters and structure should be self-adapting on-line. In addition, it is non-parametric.
- *Speed and memory efficiency*: The algorithm should be computationally effective so it can be applied to online real-time (or near real-time) applications.

In order to meet all these requirements, we propose a novel and efficient classifier named TEDAClass in which the novel TEDA framework [2] is used within a system of evolving fuzzy AnYa-type rules [6]. This combination enables to build easily interpretable model for classification. The most important contribution of this paper is the novel classifier TEDAClass. In addition, we also present the derivations of incremental calculation formulae, which are necessary for updating the incremental fuzzy rules of the classifier. The novel classifier has been successfully tested on several benchmark problems.

The reminder of the paper is organised as follows. Section 2 describes the related works. Section 3 gives a brief description of the previously proposed TEDA approach. Section 4 contains necessary novel derivations of the incremental formulae within TEDA framework, necessary for further algorithm description, and Section 5 formulates the classifier itself. The experimental data is presented in Section 6. Finally, Section 7 contains future work and concluding remarks.

2. Related work

There are many different well-studied algorithms of classification. The most relevant algorithms include Support Vector Machines (SVMs) [44], Neural Networks (Single- and Multi-layer neural networks [12,39], Radial Basis Function networks [14]), Bayesian networks [37], fuzzy rule-based models [7] or decision trees [38]. But from a wide variety of classification methods, we particularly pay attention to fuzzy logic approaches, which offer well-developed apparatus for evolving system development [8]. This kind of methods gives an essential contribution to this article. Here we review the state-of-the art algorithms in the prism of evolving systems development.

SVMs [44] provide rigorous and sparse solution for various classification problems, and are especially well renowned for the results obtained via the so-called kernel trick. They can also be implemented incrementally. If we discard some of the support vectors according to some rule, it is possible to obtain Evolving Systems based on SVM.

The evolving systems use a complex and varying combination of simple models which are adjustable “from scratch”, and capable of changing its own structure during the system learning [9,26,41]. Many evolving fuzzy systems (EFSs) [31] and evolving connectionist systems (ECSs) [27] have been developed during the last two decades. Amongst the branch of evolving fuzzy classifiers, which is a part of evolving fuzzy systems, the following methods can be mentioned: *eClass* [7], *AutoClass* [10], DEC [11], FLEXFISClass [5,32], self-evolving classifiers [19]. Also, several important works [30,33,36] has appeared in a few recent years.

Evolving Spiking Neural Networks [28,42] is one of the evolving approaches based on Artificial Neural Networks. However, reasonable efforts should be usually undertaken to learn it properly without falling into a local extremum (for example, in case of [28], the genetic algorithms are utilised for the optimisation).

Nowadays, fuzzy rule-based evolving systems are predominantly of the type of Takagi-Sugeno [43] or Mamdani [35]:

Mamdani:

$$\text{IF } (\text{ant}_i(\vec{x})) \text{ THEN } (y_i \text{ IS } Y_i), \quad (1)$$

Takagi-Sugeno:

$$\text{IF } (\text{ant}_i(\vec{x})) \text{ THEN } (y_i = \vec{x}^T \Theta_i), \quad (2)$$

where y_i is an outcome of the i th rule, $i \in [1 \dots N]$, Θ_i is a design matrix for linear regression, $\vec{x}$ is an input data vector, $\text{ant}_i(\vec{x})$ is the antecedent of the fuzzy rule.

In both cases, the antecedent is expressed as

$$\text{ant}_i(\vec{x}) : x^1 \text{ IS } L_i^1 \text{ AND } x^2 \text{ IS } L_i^2 \text{ AND } \dots \text{ AND } x^n \text{ IS } L_i^n, \quad (3)$$

where n is a dimensionality of the vector $\vec{x} = (x^1, x^2, \dots, x^n)$, and $\vec{L}_i = (L_i^1, L_i^2, \dots, L_i^n)$ is a reference vector for the fuzzy rule, where each component of the vector represents a linguistic term of the fuzzy set.

In the approach that we present in this paper, we use a more general approach which is called AnYa [6]:

AnYa:

$$\text{IF } (\vec{x} \text{ IS LIKE } \vec{L}_i) \text{ THEN } (y_i = \vec{x}^T \Theta_i), \quad (4)$$

Here the *LIKE*-predicate is not factorised, but provides a vector conditions for the whole term. This approach gives more compact, non-parametric form and does not require to consider each of the dimensions separately and pre-defining a membership function for each one of them.

The classification approach proposed in this paper is based on recently described TEDA framework [2]. In this classifier, we combine TEDA with fuzzy logic approach, which is a computationally cheap alternative to the traditional frequentistic probability theory and gives a systematic methodology. In addition, the proposed classifier gives a simple and transparent interpretation of the properties of the data sets. Finally, it can be easily combined with fuzzy rules, as it gives easy interpretable statistical metrics for data.

3. A general introduction to TEDA approach

In this section, we briefly describe the TEDA framework [2], which gives a new systematic approach to a “per point” online data analysis without making unrealistic assumptions.

Let us have some feature space $\mathfrak{X} \subseteq \mathbb{R}^n$, which we consider to be a space of data samples, or observations. For this space, we can define some distance $d(\vec{x}, \vec{y})$ (i.e. Euclidean, Mahalanobis [21], L^1 , cosine or any other convenient for the case) between feature space elements $\vec{x}, \vec{y}$.

Then, let us consider the data samples as an ordered sequence

$$\{\vec{x}_1, \vec{x}_2, \dots, \vec{x}_k, \dots\}, \vec{x}_i \in \mathbb{R}^n, \quad i \in \mathbb{N}. \quad (5)$$

where index k 's physical meaning, typically, is the instant of time when the data sample has arrived. For the sake of simplicity, index k will be referred to as time instant further.

For the whole data set/stream, we can define sum distance to some particular point $\vec{x} \in \mathfrak{X}$ for each element up to the k th one:

$$\pi^k(\vec{x}) = \sum_{i=1}^k d(\vec{x}, \vec{x}_i). \quad (6)$$

The eccentricity at the time instant k can be defined as [2]

$$\xi^k(\vec{x}) = \frac{\sum_{i=1}^k d(\vec{x}, \vec{x}_i)}{\sum_{i=1}^k \sum_{j=i+1}^k d(\vec{x}_i, \vec{x}_j)} = 2 \frac{\sum_{i=1}^k d(\vec{x}, \vec{x}_i)}{\sum_{i=1}^k \sum_{j=1}^k d(\vec{x}_i, \vec{x}_j)} = \frac{2\pi^k(\vec{x})}{\sum_{i=1}^k \pi^k(\vec{x}_i)}. \quad (7)$$

Here $k \geq 2$, $\sum_{i=1}^k \pi^k(\vec{x}_i) > 0$.

As a complement to the eccentricity, the typicality is also defined as [4]:

$$\tau^k(\vec{x}) = 1 - \xi^k(\vec{x}). \quad (8)$$

The eccentricity and typicality are both bounded [2]:

$$0 \leq \xi^k(\vec{x}) \leq 1, \quad \sum_{i=1}^k \xi^k(\vec{x}_i) = 2, \quad (9)$$

$$0 \leq \tau^k(\vec{x}) \leq 1, \quad \sum_{i=1}^k \tau^k(\vec{x}_i) = k - 2, \quad k \geq 2, \quad \sum_{i=1}^k \pi^k(\vec{x}_i) > 0. \quad (10)$$

Normalised eccentricity and typicality can also be defined as [4]:

$$0 \leq \zeta^k(\vec{x}) \leq 1, \quad \sum_{i=1}^k \zeta^k(\vec{x}_i) = 1, \quad k \geq 2, \quad (11)$$

$$t^k(\vec{x}) = \frac{\tau^k(\vec{x})}{k - 2}, \quad \sum_{i=1}^k t^k(\vec{x}_i) = 1, \quad k > 2, \quad (12)$$

Typicality and eccentricity can be easily calculated recursively that gives significant benefits for online data processing. They are exact and do not require the very restrictive assumptions which are made for the traditional probability theory, yet they have very similar properties to it as well as to the fuzzy set theory. We concentrate on Euclidean and Mahalanobis [34] distances especially, but it does not impact the generalisation of the method to other metrics. The calculations using these distances are exact and do not require the very restrictive assumptions which are made for the traditional probability theory. In addition, they have very similar properties as well as to the fuzzy sets theory. In the next two sections we will describe how our proposed TEDAClass algorithm works taking into account these two distances.

4. TEDA calculations

In this section, we describe in the detail the process to calculate the eccentricity and typicality recursively using Euclidean and Mahalanobis distances (Sections 4.1 and 4.2). In the case of Mahalanobis distance, we also describe in detail how to calculate it recursively via recursive covariance matrix and mean calculation (Section 4.3).

4.1. TEDA calculation: Euclidean distance case

Let us assume that the distance is Euclidean, and then derive some formulae describing the process of recursive calculation of the eccentricity and typicality [1]:

$$\begin{aligned}
 \xi^k(\bar{x}) &= 2 \frac{\sum_{i=1}^k d(\bar{x}, \bar{x}_i)}{\sum_{i=1}^k \sum_{j=1}^k d(\bar{x}_i, \bar{x}_j)} = 2 \frac{\sum_{i=1}^k (\bar{x}_i - \bar{x})^T (\bar{x}_i - \bar{x})}{\sum_{i=1}^k \sum_{j=1}^k (\bar{x}_i - \bar{x}_j)^T (\bar{x}_i - \bar{x}_j)} \\
 &= \left\{ \bar{\mu}_x^k = \sum_{i=1}^k \frac{\bar{x}_i}{k}, \mu_{\bar{x}^T \bar{x}}^k = \sum_{i=1}^k \bar{x}_i^T \bar{x}_i, \{\sigma_x^k\}^2 = \sum_{i=1}^k \frac{(\bar{x}_i - \bar{\mu}_x^k)^T (\bar{x}_i - \bar{\mu}_x^k)}{k} \right\} \\
 &= \frac{2k(\mu_{\bar{x}^T \bar{x}}^k - 2[\bar{\mu}_x^k]^T \bar{x} + \bar{x}^T \bar{x})}{k^2(2\mu_{\bar{x}^T \bar{x}}^k - 2[\bar{\mu}_x^k]^T \bar{\mu}_x^k)} = \{\text{using variance properties}\} \\
 &= \frac{2k(\mu_{\bar{x}^T \bar{x}}^k - 2[\bar{\mu}_x^k]^T \bar{x} + \bar{x}^T \bar{x})}{2k^2[\sigma_x^k]^2} = \frac{[\sigma_x^k]^2 + [\bar{\mu}_x^k]^T \bar{\mu}_x^k - 2[\bar{\mu}_x^k]^T \bar{x} + \bar{x}^T \bar{x}}{k[\sigma_x^k]^2} = \frac{[\sigma_x^k]^2 + (\bar{\mu}_x^k - \bar{x})^T (\bar{\mu}_x^k - \bar{x})}{k[\sigma_x^k]^2}. \quad (13)
 \end{aligned}$$

In summary,

$$\xi^k(\bar{x}) = \frac{1}{k} + \frac{(\bar{\mu}_x^k - \bar{x})^T (\bar{\mu}_x^k - \bar{x})}{k[\sigma_x^k]^2}. \quad (14)$$

Mean $\bar{\mu}_x^k$ and variance σ_x^k can be calculated recursively:

$$\bar{\mu}_x^k = \frac{k-1}{k} \bar{\mu}_x^{k-1} + \frac{\bar{x}^k}{k}, \quad k \geq 1, \quad \bar{\mu}_x^0 = \vec{0}, \quad (15)$$

$$\mu_{\bar{x}^T \bar{x}}^k = \frac{k-1}{k} \mu_{\bar{x}^T \bar{x}}^{k-1} + \frac{[\bar{x}^k]^T \bar{x}^k}{k}, \quad k \geq 1, \quad \mu_{\bar{x}^T \bar{x}}^0 = 0, \quad (16)$$

$$[\sigma_x^k]^2 = \mu_{\bar{x}^T \bar{x}}^k - [\bar{\mu}_x^k]^T \bar{\mu}_x^k. \quad (17)$$

The typicality, similarly, can be represented as

$$\tau^k(\bar{x}) = 1 - \xi^k(\bar{x}) = \frac{k-1}{k} - \frac{(\bar{\mu}_x^k - \bar{x})^T (\bar{\mu}_x^k - \bar{x})}{k[\sigma_x^k]^2}. \quad (18)$$

Then one can give an interpretation of normalised typicality and eccentricity and prove that the well-known Chebyshev inequality can be derived as a special case of TEDA if used for anomaly detection [1].

One can obtain the following inequalities for typicality:

$$t^k(\bar{x}) = \frac{1}{k-2} - \frac{1}{k(k-2)} - \frac{(\bar{\mu}_x^k - \bar{x})^T (\bar{\mu}_x^k - \bar{x})}{k(k-2)[\sigma_x^k]^2} > \frac{1}{k}, \quad (19)$$

$$\frac{k-1}{k(k-2)} - \frac{(\bar{\mu}_x^k - \bar{x})^T (\bar{\mu}_x^k - \bar{x})}{k(k-2)[\sigma_x^k]^2} > \frac{1}{k}, \quad (20)$$

$$\frac{1}{k} + \frac{1}{k(k-2)} - \frac{(\bar{\mu}_x^k - \bar{x})^T (\bar{\mu}_x^k - \bar{x})}{k(k-2)[\sigma_x^k]^2} > \frac{1}{k}, \quad (21)$$

$$\frac{1}{k(k-2)} > \frac{(\bar{\mu}_x^k - \bar{x})^T (\bar{\mu}_x^k - \bar{x})}{k(k-2)[\sigma_x^k]^2}, \quad (22)$$

$$(\bar{\mu}_x^k - \bar{x})^T (\bar{\mu}_x^k - \bar{x}) < [\sigma_x^k]^2, \quad (23)$$

The typicality can be interpreted in the following way. We see that supposing normalised typicality be greater than $1/k$ the results are shown to be exactly the same as those of the Chebyshev inequality for “ $m\sigma$ ” rule. Moreover, it is easy to check, that for “ $m\sigma$ ” rule we have:

$$t^k(\bar{x}) = \frac{1}{k-2} - \frac{1}{k(k-2)} - \frac{(\bar{\mu}_x^k - \bar{x})^T (\bar{\mu}_x^k - \bar{x})}{k(k-2)[\sigma_x^k]^2} > \frac{-n^2 + k - 1}{k(k-2)}. \quad (24)$$

Then, in order to normalise eccentricity we can check the similar equations, leading us also to Chebyshev inequality:

$$(\bar{\mu}_x^k - \bar{x})^T (\bar{\mu}_x^k - \bar{x}) < m^2 [\sigma_x^k]^2, \quad (25)$$

$$\frac{(\bar{\mu}_x^k - \bar{x})^T (\bar{\mu}_x^k - \bar{x})}{2k [\sigma_x^k]^2} < \frac{m^2}{2k}, \quad (26)$$

$$\zeta^k(\bar{x}) = \frac{1}{2k} + \frac{(\bar{\mu}_x^k - \bar{x})^T (\bar{\mu}_x^k - \bar{x})}{2k [\sigma_x^k]^2} < \frac{m^2 + 1}{2k}, \quad (27)$$

4.2. TEDA calculation: Mahalanobis distance

For cases involving high dimensional data sets (e.g., image feature mapping), the “3 σ ” rule for the Euclidean distance can result in a low accuracy. In these cases, more sophisticated dependencies should be used, enabling control over the spread of the fuzzy rules. Similarly to the case with using Euclidean distance, we can use Mahalanobis distance [34] for TEDA calculation. Then we have:

$$\xi^k(\bar{x}) = \frac{2 \sum_{i=1}^k d(\bar{x}_i, \bar{x})}{\sum_{i=1}^k \sum_{j=1}^k d(\bar{x}_i, \bar{x}_j)} = \frac{2 \sum_{i=1}^k (\bar{x} - \bar{x}_i)^T [\Sigma_x^k]^{-1} (\bar{x} - \bar{x}_i)}{\sum_{i=1}^k \sum_{j=1}^k (\bar{x}_j - \bar{x}_i)^T [\Sigma_x^k]^{-1} (\bar{x}_j - \bar{x}_i)}, \quad (28)$$

where covariance matrix is defined as follows:

$$\Sigma_x^k = \frac{1}{k} \sum_{m=1}^k (\bar{x}_m - \bar{\mu}_x^k)(\bar{x}_m - \bar{\mu}_x^k)^T. \quad (29)$$

Let us then consider the denominator and simplify it further:

$$\begin{aligned} \sum_{i=1}^k \sum_{j=1}^k d(\bar{x}_i, \bar{x}_j) &= \sum_{i=1}^k \sum_{j=1}^k (\bar{x}_j - \bar{x}_i)^T [\Sigma_x^k]^{-1} (\bar{x}_j - \bar{x}_i). \\ \sum_{i=1}^k \sum_{j=1}^k d(\bar{x}_i, \bar{x}_j) &= 2 \sum_{i=1}^k \sum_{j=1}^k \bar{x}_i^T [\Sigma_x^k]^{-1} \bar{x}_j - 2 \sum_{i=1}^k \sum_{j=1}^k \bar{x}_i^T [\Sigma_x^k]^{-1} \bar{x}_j \\ &= 2k \sum_{i=1}^k \bar{x}_i^T [\Sigma_x^k]^{-1} \bar{x}_i - 2k^2 [\bar{\mu}_x^k]^T [\Sigma_x^k]^{-1} \bar{\mu}_x^k \\ &= 2k \sum_{i=1}^k \bar{x}_i^T \left[\frac{1}{k} \sum_{m=1}^k (\bar{x}_m - \bar{\mu}_x^k)(\bar{x}_m - \bar{\mu}_x^k)^T \right]^{-1} \bar{x}_i - 2k^2 [\bar{\mu}_x^k]^T \left[\frac{1}{k} \sum_{m=1}^k (\bar{x}_m - \bar{\mu}_x^k)(\bar{x}_m - \bar{\mu}_x^k)^T \right]^{-1} \bar{\mu}_x^k \\ &= \sum \left[2k \left(\sum_{i=1}^k \bar{x}_i \bar{x}_i^T \right) \otimes \left(\frac{1}{k} \sum_{j=1}^k \bar{x}_j \bar{x}_j^T - \bar{\mu}_x^k [\bar{\mu}_x^k]^T \right)^{-1} \right] \\ &\quad - \sum \left[2k^2 (\bar{\mu}_x^k [\bar{\mu}_x^k]^T) \otimes \left(\frac{1}{k} \sum_{j=1}^k \bar{x}_j \bar{x}_j^T - \bar{\mu}_x^k [\bar{\mu}_x^k]^T \right)^{-1} \right] = \left\{ \mu_{\bar{x}\bar{x}^T} = \frac{1}{k} \sum_{j=1}^k \bar{x}_j \bar{x}_j^T \right\} \\ &= 2k^2 \sum \left(\left\{ \bar{\mu}_{\bar{x}\bar{x}^T} - \bar{\mu}_x^k [\bar{\mu}_x^k]^T \right\} \otimes \left\{ \bar{\mu}_{\bar{x}\bar{x}^T} - \bar{\mu}_x^k [\bar{\mu}_x^k]^T \right\}^{-1} \right) = \text{thanks to the covariance matrix simmetricity} \\ &= 2k^2 \sum \left(\left\{ \mu_{\bar{x}\bar{x}^T} - \bar{\mu}_x^k [\bar{\mu}_x^k]^T \right\} \left\{ \mu_{\bar{x}\bar{x}^T} - \bar{\mu}_x^k [\bar{\mu}_x^k]^T \right\}^{-1} \right) \otimes I_{n \times n} = 2k^2 \text{tr}(I_{n \times n}) = 2k^2 n. \end{aligned} \quad (31)$$

Here $\otimes$ is an element-wise matrix multiplication operator, and Σ denotes the sum of all the matrix elements.

Thus one can see that the denominator can be reduced to a constant. It gives us a reason to interpret typicality as a quantity proportional to Mahalanobis distance, and express typicality and eccentricity concepts in terms of Mahalanobis distance.

Then we can obtain:

$$\begin{aligned} \xi_k(\bar{x}) &= \frac{\sum_{i=1}^k (\bar{x} - \bar{x}_i)^T [\Sigma_x^k]^{-1} (\bar{x} - \bar{x}_i)}{k^2 n} = \frac{\bar{x}^T [\Sigma_x^k]^{-1} \bar{x} - 2 [\bar{\mu}_x^k]^T [\Sigma_x^k]^{-1} \bar{x} + [\bar{\mu}_x^k]^T [\Sigma_x^k]^{-1} \bar{\mu}_x^k}{kn} \\ &\quad + \frac{\sum_{i=1}^k \bar{x}_i^T [\Sigma_x^k]^{-1} \bar{x}_i - k [\bar{\mu}_x^k]^T [\Sigma_x^k]^{-1} \bar{\mu}_x^k}{k^2 n} = \frac{\bar{x}^T [\Sigma_x^k]^{-1} \bar{x} - 2 [\bar{\mu}_x^k]^T [\Sigma_x^k]^{-1} \bar{x} + [\bar{\mu}_x^k]^T [\Sigma_x^k]^{-1} \bar{\mu}_x^k}{kn} + \frac{1}{k} \\ &= \frac{(\bar{x} - \bar{\mu}_x^k)^T [\Sigma_x^k]^{-1} (\bar{x} - \bar{\mu}_x^k)}{kn} + \frac{1}{k}. \end{aligned} \quad (32)$$

Finally we can state the equations, analogous to Chebyshev inequality:

$$(\bar{x} - \bar{\mu}_x^k)^T [\Sigma_x^k]^{-1} (\bar{x} - \bar{\mu}_x^k) < m^2, \quad (33)$$

where m is a threshold, defining the scale of the ellipse given by the equation.

Then

$$\zeta^k(\bar{x}) = \frac{(\bar{x} - \bar{\mu}_x^k)^T [\Sigma_x^k]^{-1} (\bar{x} - \bar{\mu}_x^k)}{2kn} + \frac{1}{2k} < \frac{m^2 + n}{2kn} \quad (34)$$

or

$$\zeta^k(\bar{x}) < \frac{m^2 + n}{2kn} \quad (35)$$

Similarly for typicality we have:

$$t^k(\bar{x}) = \frac{1}{k-1} \left(1 - \frac{(\bar{x} - \bar{\mu}_x^k)^T [\Sigma_x^k]^{-1} (\bar{x} - \bar{\mu}_x^k)}{kn} - \frac{1}{kn} \right) > \frac{(k-1)n - m^2}{kn(k-1)}. \quad (36)$$

Therefore, we can specify a threshold for Mahalanobis distance, as well as calculate it recursively via recursive covariance matrix and mean calculation.

4.3. Recursive covariance matrix update

In this section, we describe other important contribution of this paper: how the recursive covariance matrix is updated in order to calculate Mahalanobis distance, recursively:

$$d^k(\bar{x}) = (\bar{x} - \bar{\mu}_x^k)^T [\Sigma_x^k]^{-1} (\bar{x} - \bar{\mu}_x^k). \quad (37)$$

We should notice here, that the distance is scalar, whilst inverse covariance matrix $[\Sigma_x^k]^{-1}$ is a square matrix $n \times n$, where n is the dimensionality of the feature space.

We can easily estimate the mean value:

$$\bar{\mu}_x^k = \frac{k-1}{k} \bar{\mu}_x^{k-1} + \frac{\bar{x}_k}{k}, \quad \bar{\mu}_x^0 = \vec{0}. \quad (38)$$

but we also need a recursive covariance matrix calculation. In what follows, we present the derivation of the method.

First, we express covariance matrix:

$$\Sigma_x^k = \frac{1}{k} \sum_{i=1}^k (\bar{x}_i - \bar{\mu}_i^k)(\bar{x}_i - \bar{\mu}_i^k)^T = \frac{1}{k} \sum_{i=1}^k \bar{x}_i \bar{x}_i^T - \bar{\mu}_x^k [\bar{\mu}_x^k]^T. \quad (39)$$

Then the recursive calculation may be applied:

$$\mu_{\bar{x}\bar{x}^T}^k = \frac{k-1}{k} \mu_{\bar{x}\bar{x}^T}^{k-1} + \frac{1}{k} \sum_{j=1}^k \bar{x}_j \bar{x}_j^T. \quad (40)$$

$$\mu_{\bar{x}^T \bar{x}}^k = \frac{k-1}{k} \mu_{\bar{x}^T \bar{x}}^{k-1} + \frac{1}{k} \sum_{j=1}^k \bar{x}_j^T \bar{x}_j. \quad (41)$$

$$\Sigma_x^k = \frac{k-1}{k} \Sigma_x^{k-1} + \frac{k^2-1}{k^2} \bar{x}_k \bar{x}_k^T + \frac{k-1}{k^2} \bar{\mu}_x^{k-1} (\bar{\mu}_x^{k-1} - 2\bar{x}_k)^T. \quad (42)$$

Thus we have derived a formula which can be used to update the covariance matrix. Then, we should derive the formulae for inverse matrix as well. Here we need to use the well-known Woodbury formula [46]:

$$(A + UBV)^{-1} = A^{-1} - A^{-1}U(B^{-1} + VA^{-1}U)^{-1}VA^{-1}. \quad (43)$$

We can use this formula twice, for the first and the second terms of the recursive expression, and then for the third one. First we calculate p^{-1} for

$$p = \frac{k-1}{k} \Sigma_x^{k-1} + \frac{k^2-1}{k^2} \bar{x}_k \bar{x}_k^T. \quad (44)$$

Here we substitute $\frac{k}{k-1} [\Sigma_x^{k-1}]^{-1}$ with A^{-1} , $B = 1$, $U = \frac{k^2-1}{k^2} \bar{x}_k$, $V = \bar{x}_{k+1}^T$. Then:

$$\begin{aligned}
p^{-1} &= \frac{k}{k-1} [\Sigma_x^{k-1}]^{-1} + \frac{k}{k-1} [\Sigma_x^{k-1}]^{-1} \frac{k+1}{k} \bar{x}_k \left(1 + \frac{k+1}{k} \bar{x}_k^T [\Sigma_x^{k-1}]^{-1} x_k \right)^{-1} \\
&\times x_k^T [\Sigma_x^{k-1}]^{-1} = \frac{k}{k-1} [\Sigma_x^{k-1}]^{-1} + \frac{\frac{k}{k-1} [\Sigma_x^{k-1}]^{-1} \frac{k+1}{k} \bar{x}_k \bar{x}_k^T [\Sigma_x^{k-1}]^{-1}}{1 + \frac{k+1}{k} \bar{x}_k^T [\Sigma_x^{k-1}]^{-1} \bar{x}_k}, \quad \Sigma_x^k = p + \frac{1}{k^2} \bar{\mu}_x^{k-1} (\bar{\mu}_x^{k-1} - 2\bar{x}_k)^T. \quad (45)
\end{aligned}$$

Here we substitute p^{-1} with A^{-1} , $B = 1$, $U = \frac{k-1}{k^2} \mu_x^{k-1}$, $V = (\mu_x^{k+1} - 2\bar{x}_k)^T$ and finally we obtain:

$$\begin{aligned}
[\Sigma_x^k]^{-1} &= p^{-1} + p^{-1} \frac{(k-1) \bar{\mu}_x^{k-1}}{k^2} \left(1 + \frac{k-1}{k^2} (\bar{\mu}_x^{k-1} - 2\bar{x}_k)^T p^{-1} \bar{\mu}_x^{k-1} \right)^{(-1)} \\
&\times (\bar{\mu}_x^{k-1} - 2\bar{x}_k)^T p^{-1} = p^{-1} + \frac{p^{-1} \frac{(k-1) \bar{\mu}_x^{k-1}}{k^2} (\bar{\mu}_x^{k-1} - 2\bar{x}_k)^T p^{-1}}{1 + \frac{(k-1)}{k^2} (\bar{\mu}_x^{k-1} - 2\bar{x}_k)^T p^{-1} \bar{\mu}_x^{k-1}} \quad (46)
\end{aligned}$$

Finally, we get:

$$[\Sigma_x^k]^{-1} = p^{-1} + \frac{p^{-1} \frac{(k-1) \bar{\mu}_x^{k-1}}{k^2} (\bar{\mu}_x^{k-1} - 2\bar{x}_k)^T p^{-1}}{1 + \frac{(k-1)}{k^2} (\bar{\mu}_x^{k-1} - 2\bar{x}_k)^T p^{-1} \bar{\mu}_x^{k-1}} \quad (47)$$

5. Our proposed TEDA classifier

In this section we explain in detail the classifier based on the TEDA framework that we propose as main contribution in this paper. This proposed classifier is presented in its two variations: based on Euclidean and Mahalanobis distances (which have been previously detailed).

As we have already explained in [Section 2](#), our proposed classifier is based on the general approach called AnYa [\[6\]](#), which is compact and non-parametric. If we describe the classifier in terms of evolving AnYa system, we obtain:

$$F = \{R_i\}, i = 1, \dots, N, R_i(\bar{x}) : \text{IF } (\bar{x} \sim \bar{x}_i^*) \text{ THEN } y_i = \bar{x}^T \Theta_i. \quad (48)$$

where F is a fuzzy rule set, each defined over one data cloud, R_i is a particular rule of this set, $y_i \in C$, $\bar{x} \in \Omega$, $F : \Omega \rightarrow C$, Ω is a feature space, C is a class space, $\bar{x}_i^*$ is cloud representative point (focal point), Θ_i is a design matrix for linear regression, $\sim$ is some relation of closeness (association/membership) of the point $\bar{x}$ to the fuzzy rule $R_i(\bar{x})$ corresponding to the data cloud. The data cloud is like a cluster, in the sense it is a group of similar and close data points, but it differs from clusters in sense that it has not any boundaries or specified shapes [\[46\]](#). For this reason, an important aspect to consider in this approach is that the clouds are described by a set of data samples but they do not have specific shape.

The set of fuzzy rules can change with each sample both quantitatively (changing focal points and design matrices) and qualitatively (fuzzy rule set cardinality fluctuations) [\[3\]](#). This aspect is essential since it ensures the evolving structure of the classifier.

The procedure of classifier learning for TEDAClass highly resembles the procedure used by *eClass* [\[38\]](#) and *AutoClass* [\[10\]](#). However, the way TEDAClass determines the closeness to the specified data cloud, as we explained below, has significantly changed. In TEDAClass, we use typicality and eccentricity concepts to calculate closeness to the rule. The more is the value of typicality, the closer is the object to the rule. In addition, for each new sample of training data, we recalculate typicality of the point. The firing strength of each rule is determined as follows:

$$w_i^k(\bar{x}) = \frac{t_i^k(\bar{x})}{\sum_{j=1}^N t_j^k(\bar{x})}, \quad (49)$$

where normalised typicality t_i^k is given over all the elements of the fuzzy rule. We assign each new coming training point to the fuzzy rule, having the largest weight. One should notice that the typicality sum is calculated over all fuzzy rules rather than over all elements of the data set over which the local typicality is calculated (it means that the weights range is from 0 to 1).

Another important aspect to consider in the proposed classifier is to define when a new rule should be created. This aspect is also an important contribution of our classifier. Contrary to the approach adopted by *eClass* [\[38\]](#) and *AutoClass* [\[10\]](#), TEDAClass does not use any global quantity (i.e., density) or all the learning data set. Instead, a new data cloud is created when the local typicality in a given instant of time for any rule, does not exceed some threshold which depends on the instant of time (corresponding to n sigma rule, for some given n). This condition is described as:

$$\forall R_i \in F t_i^k(\bar{x}) < T(k) \Leftrightarrow \eta(k) = 1. \quad (50)$$

Here

$$T(k) = \frac{(k-1)n - m^2}{kn(k-1)}$$

for Mahalanobis distance (using the same notation as in Section 4.2), and

$$T(k) = \frac{-n^2 + k - 1}{k(k - 2)}$$

for Euclidean distance. Here $\eta(k) \in \{0, 1\}$ is a flag, raised, if the cluster should be created. Local typicality is defined in the same way but the summation is over the data points of a particular data cloud only [2].

After creating a new rule by the classifier, it is necessary to delete any of the already existing rules that are closer than the same threshold. This condition is defined as:

$$R_d = \{R_i \in F : t_i^k(\bar{x}_k) > T(k)\}. \quad (51)$$

In fact, in order not to lose the previously accumulated data, the rules are not simply removed, but merge the data clouds together. For this purpose, for Euclidean and Mahalanobis distances, the means for the data clouds with indices i, j , which are being merged on the stage k , are updated as:

$$\bar{\mu}_{x,i}^k = \frac{\text{Support}(R_i)}{\text{Support}(R_i) + \text{Support}(R_j)} \bar{\mu}_{x,i}^k + \frac{\text{Support}(R_j)}{\text{Support}(R_i) + \text{Support}(R_j)} \bar{\mu}_{x,j}^k. \quad (52)$$

The covariance matrices are updated using the Woodbury formula with the formulae analogous to Section 4.3.

As a summary, and taking into account the aspects previously explained, the formal description of the algorithms (training stage and recognition stage) is given below. It is important to emphasise again that the algorithm formulation is incremental and online, and it works sequentially from scratch, sample by sample.

Formal algorithm description – training stage:

1. Initialisation: $F = \emptyset, k = 1$
2. For each new sample $\bar{x}_k$ do:
3. Calculate data sample local (per data cloud) normalised typicality $t_i^k(\bar{x}_k), i = [1 \dots N]$, N is the rule count for all the data samples.
4. $\forall R_i \in F, t_i^k(\bar{x}) < T(k)$ create cloud with a centre in $\bar{x}_k$, initialise the parameters of the local typicality incremental learning (mean $\mu_{x,N+1}^k = \bar{x}_k$ and covariance $\Sigma_{x,N+1}^k$ as identity matrix, $N = N + 1$).
5. Calculate firing strength for every fuzzy rule according to formula (49).
6. Increment the number of samples associated with the data, with respective fuzzy rule R_i with the highest typicality $t_i^k(\bar{x})$, calculated using formula (36).
7. Update the local typicality parameters for the winning data cloud, $R_i(\bar{\mu}_{x,i}^k, \Sigma_{x,i}^k)$
8. Update the parameters matrix θ_i for the rule with the highest typicality $t_i^k(\bar{x})$ by the (fuzzily weighted) RLS algorithm [38]
9. Merge the close fuzzy rules using criterion (51) and update Eq. (52).
10. Wait for a new sample and return to stage 2.

Formal algorithm description – recognition stage:

1. Initialisation: $F = \emptyset, k = 1$
2. For each new sample $\bar{x}_k$ do:
3. Calculate data sample local normalised typicality $t_i^k(\bar{x}_k), i = [1 \dots N]$, N is the rule count for all the data samples, using formula (36).
4. Calculate membership functions for each of the fuzzy rules according to formula (49).
5. Calculate the regression $\hat{y}_i = \bar{x}^k \Theta_i$ over $\bar{x}^k$ for each fuzzy rule and calculate the weighted sum $\hat{y} = w_i^k(x) \hat{y}_i$
6. Take the winning class value for the weighted sum by using the “winner takes all” approach
7. Wait for sample $\bar{x}^{k+1}$ to arrive

Finally, note that training and recognition stages can follow each other in an on-line implementation. In addition, since TEDAClass is recursive and one pass, it can be applied in many different real-time applications.

6. Experiments

To give an evidence of the proposed algorithm performance, we have conducted several experiments with very different data sets. It should be noted that although this algorithm is designed to be used in real-time, we have used several dataset to have comparable results with other classifiers. But this algorithm is totally developed to be applied in real-time.

6.1. DataSets

A. DataSets The first dataset, which is related with the symbol image recognition, is the well-known MNIST[29] dataset (Fig. 1). This is a handwritten numeral database which contains 60,000 training and 10,000 test images of 10 handwritten digits (0–9) with 28×28 pixels.

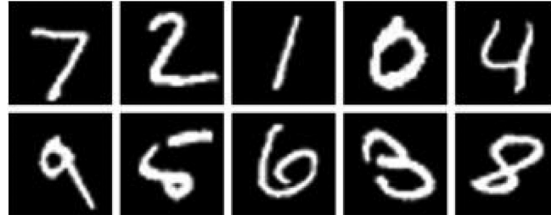


Fig. 1. Experimental data samples (MNIST).

Table 1

Recognition results comparison for MNIST[29] database.

Training set size	Classifiers and classification rate			
	<i>eClass1</i>	<i>AutoClass1</i>	<i>Neocognitron</i>	TEDAClass
500	93.26%	94.53%	92.36%	95.18%
1000	86.54%	95.82%	94.42%	95.92%
2000	96.42%	96.44%	96.04%	96.70%
3000	96.55%	96.50%	96.34%	96.67%
4000	96.62%	96.68%	96.62%	96.88%
5000	96.85%	96.91%	96.94%	97.16%
10,000	97.19%	97.24%	–	97.38%
20,000	97.32%	97.38%	–	97.53%
30,000	97.46%	97.44%	–	97.68%
40,000	97.45%	97.42%	–	97.66%
50,000	97.46%	97.38%	–	97.65%
60,000	97.46%	97.42%	–	97.63%

In addition, the method was checked on several datasets from various domains:

1. MAGIC data set [13,22,23,40], a simulation data set for registration of high energy-particles;
2. ISOLET data set [18,20,21,24], containing the vectors of features for isolated letters pronunciation for various speakers.
3. Poker hand data set [16,17] for classification of the poker hands content. The data set is particularly interesting, because it contains integer data rather than real.

6.2. Results

To evaluate the performance of TEDAClass, it is compared with the following well-known classifiers:

- Neocognitron neural network [25];
- Fuzzy evolving classifier *eClass1* [38];
- Fuzzy evolving classifier *AutoClass1* [10].

We can see that the proposed classifier is also compared with the previously proposed evolving methods *eClass1* and *AutoClass1* to make sure that it performs well against evolving methods.

Table 1 and Fig. 2 show the classification rate used to evaluate the accuracy of the four classifiers for the MNIST [29] dataset. As one can see, different sizes of training sets (from 500 to 60,000 samples) have been used. To encode the output class labels, the binary label encoding was used.

According to this data, we can see that proposed algorithm TEDAClass demonstrates significantly stronger results, achieving reasonable performance even for small amount of training data. TEDAClass algorithm, as well as *AutoClass1*, give more stable result, comparing with *eClass1*. The falls in performance of *eClass1* (i.e., for training set of 30,000 samples gives better results than one of 40,000 samples) can be explained by re-creation of some of the fuzzy rules, so that they did not have enough time to learn the patterns.

On the other hand, Table 2 shows the performance of TEDAClass on three benchmarking datasets (explained in the previous subsection). In this case, we can observe that if we take into account the MAGIC and Poker hand data sets, such modest results are caused by not so good separability of the data set. As it is described in [15], the goal of the dataset was to overcome threshold of accuracy 0.50122. The algorithm described there were tested on Waikato environment for knowledge analysis, and the best predictive accuracy, 0.56616, was given by *JRip* algorithm [45]. However, using TEDAClass, we obtain an accuracy of 0.5176.

7. Conclusions and future work

In this paper a novel and promising approach is described for classification. This approach is based on the novel TEDA framework, giving a new systematic method to the online data analysis, and AnYa-type fuzzy rules. The description of this

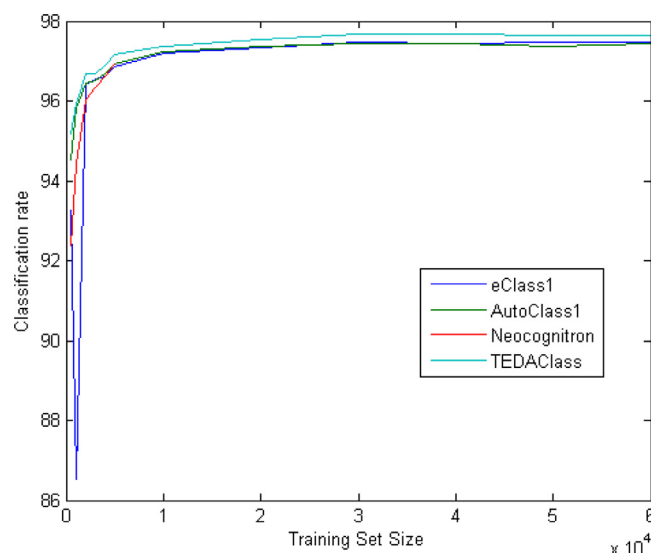


Fig. 2. Classification results graph.

Table 2

Performance of TEDAClass on some benchmarking datasets.

	Data sets		
	MAGIC	ISOLET	Poker hand
Number of the classes	2	26	10
Data set size	19,020	7797	1,025,010
Recognition rate for the data set	0.7773	0.9724	0.5016
Training data set size	9510	2599	25,010
Testing data set size	9510	5198	1,000,000
Classification rate	0.7812	0.9007	0.5176

approach has been presented and the necessary novel derivations of the incremental formulae within TEDA framework have been also explained in detail.

The approach is evolving and fully recursive, and it has shown strong results against several well-known classification algorithms (neural networks, other fuzzy rule based systems). Unlike several previously proposed algorithms (*eClass*, *AutoClass*) it does not have any global metrics for the clusters set (i.e. global density), but relies only on the local metrics of the cluster. The experiments gave an evidence of practical applicability of the algorithm for classification in different domains: symbol recognition, isolated letters pronunciation, poker combinations, γ -particles registration event.

For all these domains, we have obtained superior or similar results to those provided by the state-of-the-art algorithms. Therefore, the algorithm can claim applicability. Moreover, as it is online and evolving, it has additional advantage to be capable of processing the data streams, and learning during the exploitation stage.

Future work will be oriented to deploying this classifier in a real environment in which the data streams is received obtained in real time. However, due to the flexibility of the proposed classifier, it can be used in different domains including banking and data mining for signal information (video, images, and sound). The new distances will be added for the textual and other domain-specific information types.

References

- [1] P. Angelov, Anomaly detection based on eccentricity analysis, in: Proceedings of IEEE Symposium on Evolving and Autonomous Learning Systems, EALS, 2014, pp. 1–8.
- [2] P. Angelov, Outside the box: an alternative data analytics framework, J. Autom. Mob. Robot. Intell. Syst. 8 (2) (2014) 29–35.
- [3] P. Angelov, D. Filev, Simpl_eTS: a simplified method for learning evolving Takagi–Sugeno fuzzy models, in: Proceedings of IEEE International Conference on Fuzzy Systems, 2005, pp. 1068–1073.
- [4] P. Angelov, D.P. Filev, N. Kasabov, Evolving Intelligent Systems: Methodology and Applications, Wiley-IEEE Press, 2010.
- [5] P. Angelov, E. Lughofer, X. Zhou, Evolving fuzzy classifiers using different model architectures, Fuzzy Sets Syst. 159 (23) (2008) 3160–3182.
- [6] P. Angelov, R. Yager, A new type of simplified fuzzy rule-based system, Int. J. Gen. Syst. 41 (2) (2012) 163–185.
- [7] P. Angelov, X. Zhou, Evolving fuzzy rule-based classifiers from data streams, IEEE Trans. Fuzzy Syst.: Spec. Issue Evol. Fuzzy Syst. 16 (6) (2008) 1462–1475.
- [8] P. Angelov, X. Zhou, F. Klawonn, Evolving fuzzy rule-based classifiers, in: Proceedings of IEEE Symposium on Computational Intelligence in Image and Signal Processing, CIISP 2007, 2007, pp. 220–225.
- [9] P.P. Angelov, Autonomous Learning Systems: From Data Streams to Knowledge in Real-time, John Wiley and Sons Ltd., 2012.

- [10] P.P. Angelov, D. Kangin, X. Zhou, D. Kolev, Symbol recognition with a new autonomously evolving classifier autotool, in: Proceedings of 2014 IEEE Conference on Evolving and Adaptive Intelligent Systems, EAIS, 2014, pp. 1–7.
- [11] R.D. Baruah, P.P. Angelov, D. Baruah, Dynamically evolving fuzzy classifier for real-time classification of data streams, in: Proceedings of IEEE International Conference on Fuzzy Systems, FUZZ-IEEE 2014, 2014, pp. 383–389.
- [12] C.M. Bishop, Neural Networks for Pattern Recognition, Oxford University Press, Inc., New York, NY, USA, 1995.
- [13] R.K. Bock, P. Savicky, MAGIC gamma telescope data set. Available at: archive.ics.uci.edu/ml/datasets/MAGIC+Gamma+Telescope. (accessed May 2016)
- [14] D.S. Broomhead, D. Lowe, Radial Basis Functions, Multi-variable Functional Interpolation and Adaptive Networks, Royal Signals and Radar Establishment, Malvern, 1988.
- [15] R. Cattral, F. Oppacher, Discovering rules in the poker hand dataset, in: Proceedings of the 9th Annual Conference on Genetic and Evolutionary Computation, GECCO'07, ACM, New York, NY, USA, 2007, p. 1870.
- [16] R. Cattral, F. Oppacher, Poker Hand data set. Available at: <https://archive.ics.uci.edu/ml/datasets/Poker+Hand>, 2007.
- [17] R. Cattral, F. Oppacher, D. Deugo, Evolutionary data mining with automatic rule generalization, Recent Advances in Computers, Computing and Communications, WSEAS Press, 2002, pp. 296–300.
- [18] R. Cole, M. Fandy, T. Dietterich, ISOLET data set. Available at: <https://archive.ics.uci.edu/ml/datasets/ISOLET>, 1994.
- [19] B.S.J. Costa, P.P. Angelov, L.A. Guedes, Fully unsupervised fault detection and identification based on recursive density estimation and self-evolving cloud-based classifier, Neurocomputing 150 (2015) 289–303.
- [20] T.G. Dietterich, G. Bakiri, Error-correcting output codes: a general method for improving multiclass inductive learning programs, in: Proceedings of AAAI-91, AAAI Press, 1991, pp. 572–577.
- [21] T.G. Dietterich, G. Bakiri, Solving multiclass learning problems via error-correcting output codes, J. Artif. Intell. Res. 2 (1995) 263–286.
- [22] J. Dvorak, P. Savicky, Softening splits in decision trees using simulated annealing, in: Adaptive and Natural Computing Algorithms, in: Volume 4431 of Lecture Notes in Computer Science, Springer, Berlin, Heidelberg, 2007, pp. 721–729.
- [23] R.K.B. et al., Methods for multidimensional event classification: a case study using images from a cherenkov gamma-ray telescope, Nucl. Instrum. Methods Phys. Res. Sect. A: Accel. Spectro. Detect. Assoc. Equip. 516 (2) (2004) 511–528.
- [24] M. Fandy, R. Cole, Spoken letter recognition, in: R. Lippmann, J. Moody, D. Touretzky (Eds.), Advances in Neural Information Processing Systems, Vol. 3, Morgan-Kaufmann, 1991, pp. 220–226.
- [25] K. Fukushima, Neocognitron for handwritten digit recognition, Neurocomputing 51 (2003) 161–180.
- [26] J. Gama, Knowledge Discovery from Data Streams, Chapman and Hall/CRC, Boca Raton, Florida, 2010.
- [27] N. Kasabov, Evolving Connectionist Systems: The Knowledge Engineering Approach, Second Edition, Springer Verlag, London, 2007.
- [28] N. Kasabov, V. Feigin, Z. Hou, Y. Chen, L. Liang, R. Krishnamurthi, M. Othman, P. Parmar, Evolving spiking neural networks for personalised modelling, classification and prediction of spatio-temporal patterns with a case study on stroke, Neurocomputing 134 (2014) 269–279.
- [29] Y. LeCun, C. Cortes, MNIST handwritten digit database. Available at: <http://yann.lecun.com/exdb/mnist/>, 2010.
- [30] A. Lemos, W. Caminhas, F. Gomide, Reliable all-pairs evolving fuzzy classifiers, Inf. Sci. 220 (2013) 64–85.
- [31] E. Lughofer, Evolving Fuzzy Systems – Methodologies, Advanced Concepts and Applications, Springer, Berlin Heidelberg, 2011.
- [32] E. Lughofer, P. Angelov, X. Zhou, Evolving single- and multi-model fuzzy classifiers with flexfis-class, in: Proceedings of FUZZ-IEEE 2007, London, UK, 2007, pp. 363–368.
- [33] E. Lughofer, O. Buchtala, Reliable all-pairs evolving fuzzy classifiers, IEEE Trans. Fuzzy Syst. 21 (4) (2013) 625–641.
- [34] P.C. Mahalanobis, On the generalized distance in statistics, Proc. Natl. Inst. Sci. 2 (1936) 49–55.
- [35] E.H. Mamdani, S. Assilian, An experiment in linguistic synthesis with a fuzzy logic controller, Int. J. Man-Mach. Stud. 7 (1) (1975) 1–13.
- [36] J. Ordóñez, J. Iglesias, P. de Toledo, A. Ledezma, A. Sanchis, Online activity recognition using evolving classifiers, Expert Syst. Appl. 40 (4) (2013) 1248–1255.
- [37] J. Pearl, Causality: Models, Reasoning, and Inference, Cambridge University Press, New York, NY, USA, 2000.
- [38] L. Rokach, O. Maimon, Data Mining with Decision Trees – Theory and Applications, Volume 69 of Series in Machine Perception and Artificial Intelligence, World Scientific, 2007.
- [39] F. Rosenblatt, The perceptron: a probabilistic model for information storage and organization in the brain, Psychol. Rev. 65 (6) (1958) 386–408.
- [40] P. Savicky, E. Kotrc, Experimental Study of Leaf Confidences for Random Forest, in: J. Antoch (Ed.), Proceedings of COMPSTAT 2004, In: Computational Statistics, Physica Verlag, Heidelberg, 2004, pp. 1767–1774.
- [41] M. Sayed-Mouchaweh, E. Lughofer, Learning in Non-Stationary Environments: Methods and Applications, Springer, New York, 2012.
- [42] S. Schliebs, N. Kasabov, Evolving spiking neural network – a survey, Evol. Syst. 4 (2) (2012) 87–98.
- [43] T. Takagi, M. Sugeno, Fuzzy identification of systems and its applications to modeling and control, IEEE Trans. Syst. Man Cybern. 15 (1) (1985) 116–132.
- [44] V.N. Vapnik, The Nature of Statistical Learning Theory, Springer-Verlag New York, Inc., New York, NY, USA, 1995.
- [45] I.H. Witten, E. Frank, M.A. Hall, Data Mining: Practical Machine Learning Tools and Techniques, Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 2011.
- [46] M.A. Woodbury, Inverting Modified Matrices, Number 42 in Statistical Research Group Memorandum Reports, Princeton University, Princeton, NJ, 1950.